TP3 : CONSOLIDATION DE LA PROGRAMMATION

BUT 2 R&T
(GROUPE 1)

UNIVERSITÉ SORBONNE PARIS NORD

Réaliser :
- Arol Tindom

Encadré par:
- M'LIK MORADTP

2025 - 2026

TABLE DES MATIERES	PAGES
INTRODUCTION	1
Exercice 1 : Gestion d'un parkin	*1 à 3*
Exercice 2 : L'algèbre d'Allen	*4 à 9*
Exercice 3 : Python et les atomes	*10 à 15*
CONCLUSION	*16*

INTRODUCTION

Ce TP s'inscrit dans le cadre du module R308 – Consolidation de la Programmation
L'objectif est de consolider les bases de la programmation orientée objet (POO) en Python :
- Concevoir et implémenter des classes et leurs méthodes
- Respecter les principes d'encapsulation, d'héritage et de réutilisabilité du code
- Tester le bon fonctionnement du programme par des scénarios réels

Les exercices portent sur :
- La simulation d'un système de gestion de parking
- L'implémentation de l'algèbre d'Allen pour raisonner sur des intervalles
- La modélisation d'atomes et l'analyse de leurs propriétés physiques et chimiques

R308 - consolidation de la programmation : TP3

Gestion d'un parking

Objectif
Mettre en place un système qui gère le nombre de places disponibles dans un parking en temps réel et qui affiche les informations sur plusieurs portails

Démarche
- Création de deux classes : Parking et Portail
- Gestion des entrées et sorties via des méthodes (entrée, sortie)
- Affichage de l'état global (places libres, nombre d'entrées/sorties par portail)
- Utilisation des assertions pour éviter les données invalides
- Conception UML de la relation Parking-Portail (composition)

Donnons les classes nécessaires pour effectuer cette simulation

Welcome 1.a.py

home > student > 905 > 12406298 > Bureau > Mes_Montages > 12406298 > TP1 programmation > 1.a.py > ...

```python
class Portail:
    """Représente un portail d'entrée/sortie du parking"""
    def __init__(self, numero):
        assert isinstance(numero, int) and numero > 0, "Numéro de portail invalide"
        self.numero = numero
        self.entrees = 0
        self.sorties = 0

    def enregistrer_entree(self):
        self.entrees += 1

    def enregistrer_sortie(self):
        self.sorties += 1

    def afficher(self, places_libres, capacite):
        """Affiche les infos de ce portail"""
        if places_libres > 0:
            msg_places = f"{places_libres}/{capacite} places libres"
        else:
            msg_places = "Parking COMPLET"
        return (f"Portail #{self.numero} | "
                f"Entrées: {self.entrees} | Sorties: {self.sorties} | {msg_places}")

```

Welcome 1.a.py

home > student > 905 > 12406298 > Bureau > Mes_Montages > 12406298 > TP1 programmation > 1.a.py > ...

```python
class Parking:
    """Classe représentant un parking avec plusieurs portails"""
    def __init__(self, adresse, capacite, nb_portails):
        assert isinstance(adresse, str) and len(adresse) > 0, "Adresse invalide"
        assert isinstance(capacite, int) and capacite > 0, "Capacité invalide"
        assert isinstance(nb_portails, int) and nb_portails > 0, "Nombre de portails invalide"

        self.adresse = adresse
        self.capacite = capacite
        self.occupes = 0  # nombre de places actuellement occupées
        self.portails = [Portail(i + 1) for i in range(nb_portails)]

    def entree(self, num_portail):
        if self.occupes < self.capacite:
            self.occupes += 1
            self.portails[num_portail - 1].enregistrer_entree()
        else:
            print("Parking complet ! Entrée refusée.")

    def sortie(self, num_portail):
        if self.occupes > 0:
            self.occupes -= 1
            self.portails[num_portail - 1].enregistrer_sortie()
        else:
            print("Parking déjà vide !")

    def places_libres(self):
        return self.capacite - self.occupes

    def afficher_etat(self):
        """Affiche l'état de chaque portail"""
        for portail in self.portails:
            print(portail.afficher(self.places_libres(), self.capacite))

```

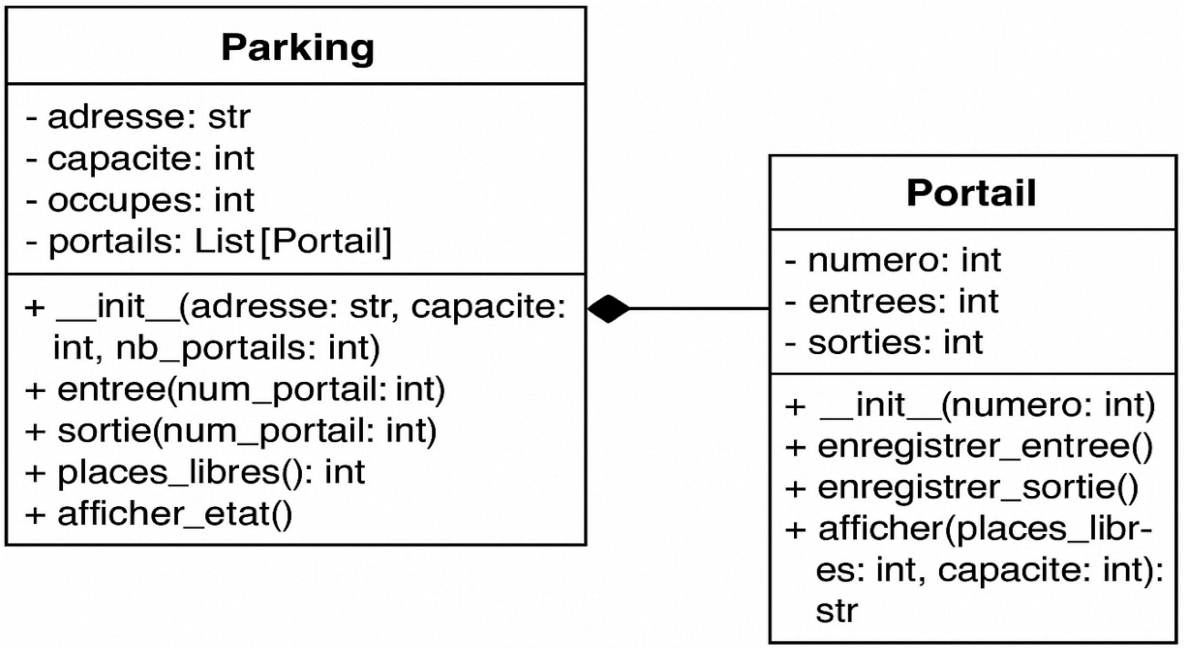
voici un exemple d'utilisation :

```python
# Création d'un parking de 10 places, avec 2 portails
p = Parking("12 rue de l'Université", 10, 2)

p.entree(1)
p.entree(2)
p.entree(1)

p.sortie(2)

p.afficher_etat()
```

voici le résultat :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    JUPYTER

Portail #1 | Entrées: 2 | Sorties: 0 | 8/10 places libres
Portail #2 | Entrées: 1 | Sorties: 1 | 8/10 places libres
12406298@r20203:~$ /bin/python3 "/home/student/905/12406298/Bureau/Mes_Montages/12406298/TP1 programmation/1.a.py"
Portail #1 | Entrées: 2 | Sorties: 0 | 8/10 places libres
Portail #2 | Entrées: 1 | Sorties: 1 | 8/10 places libres
```

<u>diagramme UML simple</u>



L'Algèbre d'Allen

Objectif

Implémenter une classe **Intervalle** pour représenter des intervalles et tester les relations de l'algèbre d'Allen

Démarche

- Création de la classe Intervalle avec vérification de validité
- Développement des méthodes : `e`, `b`, `m`, `o`, `s`, `f`, `d`
- Implémentation de `bi` en réutilisant `b` (inverse)
- Écriture d'un programme de test qui vérifie les 13 relations

1- Donner la méthode constructeur de la classe Intervalle. On rappelle que la méthode constructeur doit seulement construire d'objets valides (conformes à la définition d'un intervalle)

Un **intervalle** est défini par deux bornes d et `f` (début et fin) avec la contrainte d≤ `f`
On doit donc :

- Vérifier que d et `f` sont des nombres (`int` ou `float`)
- Vérifier que d ≤ `f`
- Sauvegarder les bornes comme attributs d'instance

programme :

```python
class Intervalle:
    """
    Représente un intervalle [d, f] avec d <= f (bornes incluses).
    Fournit les relations de l'algèbre d'Allen entre self (X) et un autre intervalle Y.
    """
    def __init__(self, d, f):
        # validation du constructeur : d et f sont numériques et d <= f
        assert (isinstance(d, (int, float)) and isinstance(f, (int, float))), \
            "Bornes doivent être des nombres"
        assert d <= f, "Borne de début doit être <= borne de fin"
        self.d = float(d)
        self.f = float(f)
```

<u>Explication</u> :

- **`assert`** permet de garantir la validité des intervalles au moment de la création
- Conversion en **`float`** pour unifier le type et éviter les problèmes entre `int` et `float`

<u>2 - définir la méthode e()qui implémente la relation égale</u>

La relation **égale (equals)** signifie que les deux intervalles ont **même début et même fin**

```python
    def e(self, Y):
        """X égal Y : mêmes bornes"""
        return (self.d == Y.d) and (self.f == Y.f)
```

<u>Explication :</u>

- On retourne **`True`** si les deux intervalles ont exactement les mêmes bornes

- C'est la relation la plus simple de l'algèbre

<u>3. Donner les codes des méthodes : b(), m(), o(), s(), f() et d()</u>

programme :

```python
    def b(self, Y):
        """X before Y : la fin de X est strictement avant le début de Y"""
        return self.f < Y.d

    def m(self, Y):
        """X meets Y : la fin de X est exactement le début de Y"""
        return self.f == Y.d

    def o(self, Y):
        """
        X overlaps Y : X commence avant Y.d, X.f est entre Y.d et Y.f (strict)
        formal: X.d < Y.d < X.f < Y.f
        """
        return (self.d < Y.d) and (Y.d < self.f) and (self.f < Y.f)

    def s(self, Y):
        """
        X starts Y : mêmes débuts, X se termine avant Y
        X.d == Y.d and X.f < Y.f
        """
        return (self.d == Y.d) and (self.f < Y.f)

    def f(self, Y):
        """
        X finishes Y : mêmes fins, X commence après Y
        X.f == Y.f and X.d > Y.d
        """
        return (self.f == Y.f) and (self.d > Y.d)

    def d(self, Y):
        """
        X during Y : X est strictement à l'intérieur de Y
        Y.d < X.d < X.f < Y.f
        """
        return (Y.d < self.d) and (self.f < Y.f)
```

<u>Explication :</u>

- Chaque relation correspond à l'un des 13 cas de l'algèbre d'Allen

- On utilise les **comparaisons strictes ou larges** en fonction de la définition

<u>4 - La relation après (notée bi) est l'inverse de la relation avant (notée b) : (Xest après Y) ⇐⇒ (Y est avant X). En profitant de cette observation proposer une implémentation de la méthode bi() en utilisant la méthode b().</u>

Donc :

`X.bi(Y)` ⇔ `Y.b(X)`

```python
    def bi(self, Y):
        """X after Y : implémenté via Y.b(X) conformément à l'énoncé"""
        return Y.b(self)

```

Explication :

- Plutôt que de dupliquer la logique de comparaison (`self.d > Y.f`), on réutilise b( ) sur l'argument inversé → **moins de duplication**, code plus clair et plus maintenable

programme entier :

Welcome | 1.a.py

home > student > 905 > 12406298 > Bureau > Mes_Montages > 12406298 > TP1 programmation > 1.a.py > Int

```python

class Intervalle:
    """ Représente un intervalle [d, f] avec d <= f (bornes incluses)"""
    def __init__(self, d, f):
        # validation du constructeur : d et f sont numériques et d <= f
        assert (isinstance(d, (int, float)) and isinstance(f, (int, float))), \
            "Bornes doivent être des nombres"
        assert d <= f, "Borne de début doit être <= borne de fin"
        self.d = float(d)
        self.f = float(f)

    def e(self, Y):
        """X égal Y : mêmes bornes"""
        return (self.d == Y.d) and (self.f == Y.f)

    def b(self, Y):
        """X before Y : la fin de X est strictement avant le début de Y"""
        return self.f < Y.d

    def m(self, Y):
        """X meets Y : la fin de X est exactement le début de Y"""
        return self.f == Y.d

    def o(self, Y):
        """
        X overlaps Y : X commence avant Y.d, X.f est entre Y.d et Y.f (strict)
        formal: X.d < Y.d < X.f < Y.f
        """
        return (self.d < Y.d) and (Y.d < self.f) and (self.f < Y.f)
```

```python
    def s(self, Y):
        """
        X starts Y : mêmes débuts, X se termine avant Y
        X.d == Y.d and X.f < Y.f
        """
        return (self.d == Y.d) and (self.f < Y.f)

    def f(self, Y):
        """
        X finishes Y : mêmes fins, X commence après Y
        X.f == Y.f and X.d > Y.d
        """
        return (self.f == Y.f) and (self.d > Y.d)

    def d(self, Y):
        """
        X during Y : X est strictement à l'intérieur de Y
        Y.d < X.d < X.f < Y.f
        """
        return (Y.d < self.d) and (self.f < Y.f)

    # --- inverses (utilisent les méthodes correspondantes sur l'autre intervalle) ---
    def bi(self, Y):
        """X after Y : implémenté via Y.b(X) conformément à l'énoncé"""
        return Y.b(self)

    def mi(self, Y):
        """X met-by Y (X is met by Y)"""
        return Y.m(self)

    def oi(self, Y):
        """X overlapped-by Y"""
        return Y.o(self)

    def si(self, Y):
        """X started-by Y"""
        return Y.s(self)

    def fi(self, Y):
        """X finished-by Y"""
        return Y.f(self)

    def di(self, Y):
        """X contains Y (inverse de during)"""
        return Y.d(self)
```

```python
    # Optionnel : représentation lisible
    def __repr__(self):
        return f"Intervalle({self.d}, {self.f})"
```

5 - Donner un programme qui permet de tester le bon fonctionnement de la classe Intervalle que vous proposez.

```python
if __name__ == "__main__":
    X = Intervalle(1, 2)
    Y = Intervalle(3, 4)
    print("X before Y ?", X.b(Y))
    print("X after Y ?", X.bi(Y))

    Y2 = Intervalle(2, 3)
    print("X meets Y2 ?", X.m(Y2))

    Y3 = Intervalle(1.5, 3)
    print("X overlaps Y3 ?", X.o(Y3))
```

résultat :

```
12406298@r20203:~$ /bin/python3 "/home/student/
X before Y ? True
X after Y ? False
X meets Y2 ? True
X overlaps Y3 ? True
```

Explication :

- Ce test couvre plusieurs relations (avant, après, rencontre, chevauchement)
- Il peut être complété avec d'autres exemples pour vérifier toutes les relations

Python et les atomes

Objectif

Modéliser des atomes en utilisant les classes Python et calculer leurs propriétés (masse, électrons externes, isotopie, famille chimique)

Démarche

- Définition des attributs : `symbole`, `A`, `Z`, `neutrons`
- Développement des méthodes :

  `masse()` : calcul de la masse totale

  `NbElecExt()` : nombre d'électrons sur la couche externe

  `memeFamille()` : comparaison avec un autre atome

  `isotope()` : vérification d'isotopie

- Tests avec plusieurs atomes (Carbone 12, Carbone 14, Oxygène 16, Néon 20)

1- Donner l'ensemble des attributs de la classe Atom en précisant pour chaque attribut les informations suivantes : Nature : attribut d'instance ou attribut de classe. Type : domaine de valeurs. Contraintes à respecter sur les valeurs de l'attribut

Attributs d'instance

Attribut	Nature	Type	Contraintes
symbole	instance	str	Lettresmajuscules/minuscules, non vide
`A` (nombre de masse)	instance	int	A > 0
`Z` (nombre atomique)	instance	int	Z > 0 et Z ≤ A
neutrons	instance	int	calculé automatiquement (`A - Z`), doit être ≥ 0

Attributs de classe

Attribut	Nature	Type	Contraintes
M_PROTON	classe	float	masse d'un proton (`1.673 × 10^-27 kg`)
M_NEUTRON	classe	float	masse d'un neutron (`1.675 × 10^-27 kg`)

Explication :

- On sépare les constantes physiques (protons/neutrons) comme **attributs de classe** car elles sont communes à tous les atomes

- Les données propres à chaque atome (`A`, `Z`, `symbole`) sont des **attributs d'instance**

2 - Donner le code de la méthode constructeur de la classe Atom

Welcome 1.a.py

home > student > 905 > 12406298 > Bureau > Mes_Montages > 12406298 > TP1 programmation > 1.a.py > Atom

```python

class Atom:
    """ Attributs de classe """

    # Attributs de classe : constantes physiques
    M_PROTON = 1.673e-27   # kg
    M_NEUTRON = 1.675e-27  # kg

    def __init__(self, symbole, A, Z):
        """
        Constructeur de la classe Atom.
        :param symbole: symbole chimique (str)
        :param A: nombre de masse (int)
        :param Z: nombre atomique (int)
        """
        assert isinstance(symbole, str) and len(symbole) > 0, "Symbole invalide"
        assert isinstance(A, int) and A > 0, "Nombre de masse A invalide"
        assert isinstance(Z, int) and Z > 0, "Nombre atomique Z invalide"
        assert Z <= A, "Z ne peut pas être supérieur à A"

        self.symbole = symbole
        self.A = A
        self.Z = Z
        self.neutrons = A - Z
```

Explication :

- Utilisation de **`assert`** pour garantir les contraintes sur les attributs

- Calcul direct du nombre de neutrons (`A - Z`) pour éviter les incohérences

3 - Donner le code de la méthode masse() qui retourne la masse totale de l'élément

La masse ≈ somme des masses des protons + neutrons (masse des électrons négligée)

```
    def masse(self):
        """Retourne la masse totale de l'atome (kg)."""
        return (self.Z * Atom.M_PROTON) + (self.neutrons * Atom.M_NEUTRON)

```

Explication :

- On multiplie le nombre de protons par la masse d'un proton et le nombre de neutrons par la masse d'un neutron

- Utilisation de **`Atom.M_PROTON`** (attribut de classe)

4 - Donner le code de la méthode NbElecExt() qui retourne le nombre d'électrons sur la couche externe

Règles :

- Première couche : max 2 électrons

- Les suivantes : max 8 électrons

- Une couche doit être remplie avant de passer à la suivante

```python
    def NbElecExt(self):
        """Retourne le nombre d'électrons sur la couche externe."""
        electrons = self.Z
        # Première couche (max 2 électrons)
        if electrons <= 2:
            return electrons
        electrons -= 2
        # Remplir les couches de 8 électrons
        while electrons > 8:
            electrons -= 8
        return electrons
```

Explication :

- On enlève d'abord 2 électrons pour remplir la première couche
- Ensuite on retire des paquets de 8 jusqu'à ce qu'on atteigne la dernière couche partielle
- On retourne le reste → nombre d'électrons sur la couche externe

5 - Donner le code de la méthode memeFamille(elem) qui teste si l'élément est de la même famille d'un autre élément elem.

```python
    def memeFamille(self, elem):
        """
        Vérifie si l'atome est dans la même famille qu'un autre élément
        """
        return self.NbElecExt() == elem.NbElecExt()
```

Explication :

- On compare simplement le résultat de **`NbElecExt()`** entre deux atomes

6 - Donner le code de la méthode isotope(elem) qui teste si l'élément est un isotope de l'élément elem

```python
    def isotope(self, elem):
        """
        Vérifie si l'atome est un isotope d'un autre élément.
        """
        return (self.Z == elem.Z) and (self.A != elem.A)

```

Explication :

- Deux atomes sont isotopes si leur nombre de protons (**Z**) est identique mais leur nombre de masse (**A**) est différent

programme entier :

```python
class Atom:
    """ Attributs de classe  """

    # Attributs de classe : constantes physiques
    M_PROTON = 1.673e-27   # kg
    M_NEUTRON = 1.675e-27  # kg

    def __init__(self, symbole, A, Z):
        """
        Constructeur de la classe Atom.
        :param symbole: symbole chimique (str)
        :param A: nombre de masse (int)
        :param Z: nombre atomique (int)
        """
        assert isinstance(symbole, str) and len(symbole) > 0, "Symbole invalide"
        assert isinstance(A, int) and A > 0, "Nombre de masse A invalide"
        assert isinstance(Z, int) and Z > 0, "Nombre atomique Z invalide"
        assert Z <= A, "Z ne peut pas être supérieur à A"

        self.symbole = symbole
        self.A = A
        self.Z = Z
        self.neutrons = A - Z

    def masse(self):
        """Retourne la masse totale de l'atome (kg)."""
        return (self.Z * Atom.M_PROTON) + (self.neutrons * Atom.M_NEUTRON)

    def NbElecExt(self):
        """Retourne le nombre d'électrons sur la couche externe."""
        electrons = self.Z
        # Première couche (max 2 électrons)
        if electrons <= 2:
            return electrons
        electrons -= 2
        # Remplir les couches de 8 électrons
        while electrons > 8:
            electrons -= 8
        return electrons
```

```python
    def memeFamille(self, elem):
        """
        Vérifie si l'atome est dans la même famille qu'un autre élément
        """
        return self.NbElecExt() == elem.NbElecExt()

    def isotope(self, elem):
        """
        Vérifie si l'atome est un isotope d'un autre élément.
        """
        return (self.Z == elem.Z) and (self.A != elem.A)

    def __repr__(self):
        """Représentation lisible de l'objet."""
        return f"Atom({self.symbole}, A={self.A}, Z={self.Z})"
```

Exemple d'utilisation :

```python
# Programme de test (main)
if __name__ == "__main__":
    # Création de quelques atomes
    carbone12 = Atom("C", 12, 6)
    carbone14 = Atom("C", 14, 6)
    oxygene16 = Atom("O", 16, 8)

    print("Masse C12 :", carbone12.masse(), "kg")
    print("Electrons couche externe C12 :", carbone12.NbElecExt())
    print("C12 et O16 même famille ?", carbone12.memeFamille(oxygene16))
    print("C12 et C14 isotopes ?", carbone12.isotope(carbone14))
```

résultat :

```
12406298@r20203:~$ /bin/python3 "/home/student/905/
Masse C12 : 2.0088000000000002e-26 kg
Electrons couche externe C12 : 4
C12 et O16 même famille ? False
C12 et C14 isotopes ? True
```

Conclusion

Ce TP a permis de :

- Mettre en pratique les concepts de **POO en Python** (constructeurs, méthodes, encapsulation)

- Simuler des situations réelles (parking, intervalles temporels, structure atomique)

- Développer de bonnes habitudes : validation des données (`assert`), séparation du code et des tests

Utilité professionnelle :

Ce TP entraîne à concevoir des systèmes modulaires et extensibles, un savoir-faire essentiel pour développer des applications robustes et maintenables dans un contexte industriel